

장진성 백엔드 포트폴리오

https://github.com/answk427/Spring_Shop_portfolio

<https://jjsdev.duckdns.org>

기술스택

Category	Tech Choice	Reason for Selection
Backend	Spring Boot, Java 21	안정적인 백엔드 구성
Database	MySQL, Redis	관계형 데이터 저장 및 RefreshToken TTL 관리
Security	Spring Security, JWT	Stateless 기반의 효율적인 인증/인가 처리
Infra / Test	Docker, JUnit	실제 환경과 유사한 테스트
Version Control	GIT, Flyway	버전 관리 / DB 구조 변경 유연화
Util	MapStruct, Swagger	Entity <-> Dto 변환 코드 간소화 / API 문서화, 테스트
배포	Docker, EC2, nginx, RDS, S3, GitHub	간편한 클라우드 서버 구축 / GitHub Actions 자동 배포

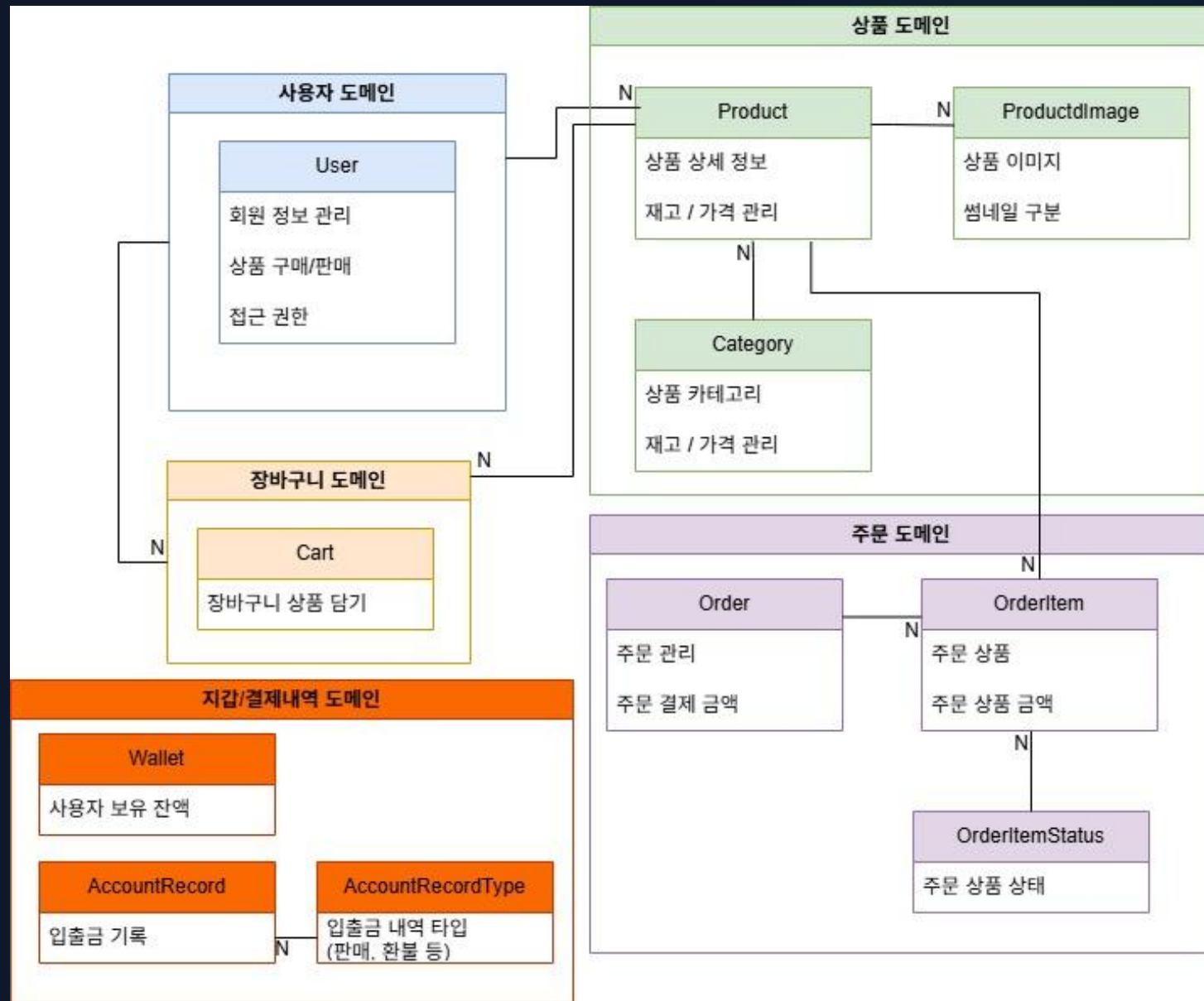
DOMAIN STRUCTURE

도메인 간 책임분리

각 도메인 관련 로직을
집중시키고 확장성 증가

상태 관리

주문 흐름을 관리하기 용이하고
명확한 기록을 남길 수 있습니다



주요 흐름



사용자



상품 탐색



장바구니



주문



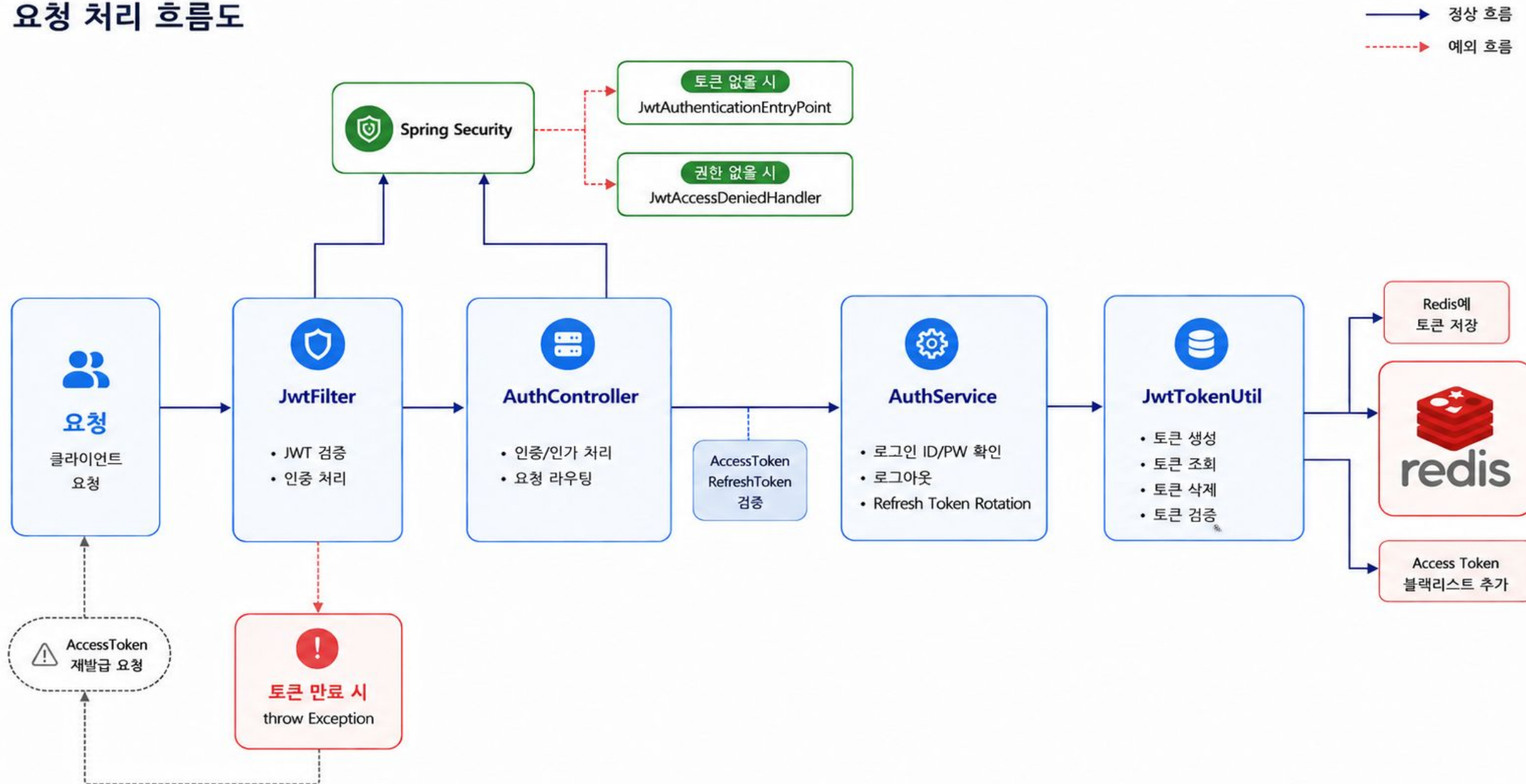
결제 (지갑)



주문 처리

AUTHENTICATION WORKFLOW

요청 처리 흐름도



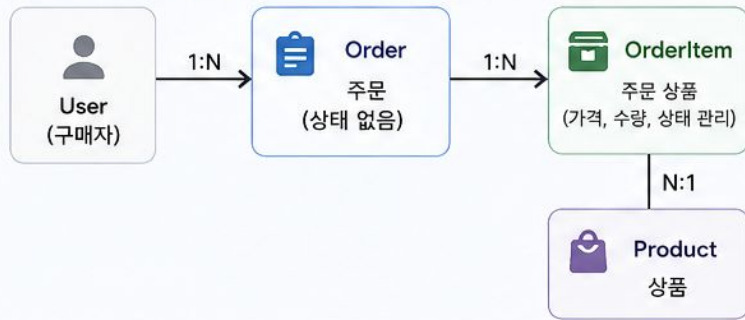
JWT 토큰을 통한 자원에 대한 접근 제어. 단순 상품 조회는 인증없이 접근 가능. 구매/판매는 인증필요

주문 과정 / 상태 관리

주문 시스템 설계 OrderItem 기반 상태 관리

주문은 여러 상품의 모음이며,
각 상품(OrderItem) 단위로 상태를 관리하고
상태 변화에 따라 결제 금액이 이동합니다.

주문 도메인 설계



OrderItem 상태 흐름



설계 포인트

- Order에는 상태를 두지 않고, OrderItem 단위로 상태를 관리
- 하나의 주문 내에서도 상품별로 서로 다른 상태를 가질 수 있음
- 상태 변화에 따라 구매자/판매자의 Wallet 잔액이 이동

결제 금액 흐름



핵심 요약

✓ OrderItem 단위 상태 관리로
유연한 주문 처리

✓ 상태 변화에 따른
결제 금액 자동 이동

✓ 환불/반품/취소도
일관된 흐름으로 처리

✓ 확장 가능한
전문 이커머스 구조

결제는 주문(Order) 단위, 배송/취소/반품 등은 주문 상품(OrderItem) 단위로 수행

| 동시성 문제 해결



100
Concurrent Users

vs



10
Available Items

비관적 락(Pessimistic Lock) 적용

충돌 빈도가 높은 상품 재고 시스템 특성을 고려하여, 재고 증감시 비관적 락을 적용해 데이터 정합성을 최우선으로 보장했습니다.

```
@Lock(LockModeType.PESSIMISTIC_WRITE) 2개 사용 위치 👤 wlstjd50  
@Query("SELECT p from Product p join fetch p.category join fetch p.seller where p.id in :ids")  
List<Product> findAllByIdWithLock(@Param("ids") List<Long> ids);
```

고려사항 : 주문 생성/취소 시 상품 수량 변경에 데이터 정합성을 위해 비관적 락 적용

쿼리 횟수를 줄이기 위해 상품ID를 List로 전달받아 한번에 조회

N+1 문제를 해결하기 위해 FETCH JOIN 사용

TroubleShooting

! 문제 상황

배경

10개의 재고가 있는 상품을 100명의 사용자가
동시에 주문을 시도하는 동시성 테스트 수행

```
(orderservice.stockConcurrencyTest())
```

현상

데이터 정합성을 위해 Pessimistic Lock을 사용했음에도
불구하고, 모든 스레드가 상품을 조회했을 때
수량이 10개에서 줄어들지 않는 현상

🔍 원인 분석 및 해결

원인 분석

JPA 영속성 컨텍스트의 동일성 보장, Pessimistic Lock의 실행순서 문제

1 사전 조회에 의한 캐싱

장바구니 조회 과정에서 Product(상품) 엔티티의 상세정보를 조회하며
해당 엔티티가 1차 캐시에 '재고 10' 상태로 로드됨.

2

FOR UPDATE 쿼리가 날아가고 최신 데이터를 가져오지만,
JPA가 1차캐시에 있는 과거 데이터를 반환함.

✅ 해결

장바구니에 담긴 ProductID만 가져오도록 수정하여
1차 캐시에 미리 올라가지 않도록 함.

동시성 문제 발생 흐름



핵심 포인트



JPA 1차 캐시의 동일성 보장 원리 이해



쿼리 실행 순서와 캐시 동작 방식 고려



불필요한 엔티티 조회 최소화로
정합성 문제 해결

동적 검색

QueryDSL

복잡한 동적 조건 검색의 확장성

안정성 및 가독성 확보

DTO Projection으로 성능향상

FullText Index

검색 최적화를 위해 FULLTEXT

인덱스 활용

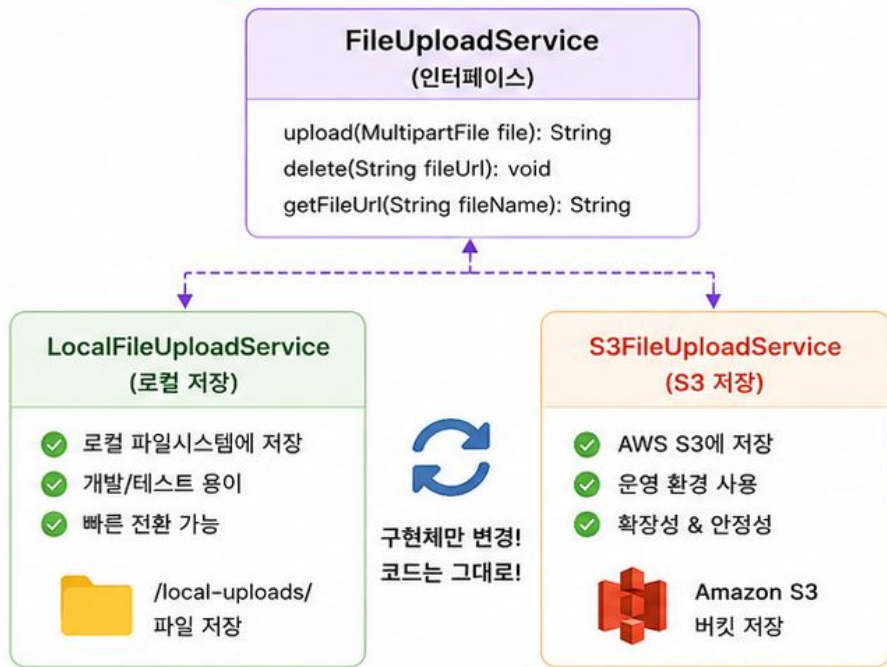
```
private Predicate matchKeyword(String keyword) { 2 usages  👤 wlstjd50
    if (!StringUtils.hasText(keyword)) {
        return null;
    }
    //TEXT FULL 인덱스를 활용해 검색
    return Expressions.numberTemplate(
        Double.class,
        template: "function('match_against', {0}, {1}, {2})",
        product.name,
        product.description,
        keyword
    ).gt(right: 0);
}

private BooleanExpression categoryIdEq(List<Long> categoryIds) {
    return categoryIds != null && !categoryIds.isEmpty() ?
        product.category.id.in(categoryIds) : null;
}
```

파일 업로드

파일 업로드 추상화 + 환경별 설정 분리로 유연하고 테스트하기 좋은 구조

1. 파일 업로드 서비스 추상화



💡 인터페이스 기반 설계로 스토리지 전환이 매우 쉽고 유연합니다.

2. 설정 파일 분리 (프로필 기반)



3. .env 파일로 민감 정보 관리



4. 유연한 테스트 환경 구성



핵심 장점

- ✓ 스토리지 전환이 코드 변경 없이 가능
- ✓ 환경별 설정 분리로 관리가 쉬움
- ✓ 민감 정보는 .env로 안전하게 관리
- ✓ 다양한 테스트 시나리오 구성 가능

파일 업로드

CloudFront

S3 접근 보안, 이미지 캐싱

S3에 파일 경로만 저장해 유연성

상승

(CloudFrontURL + 경로) 반환

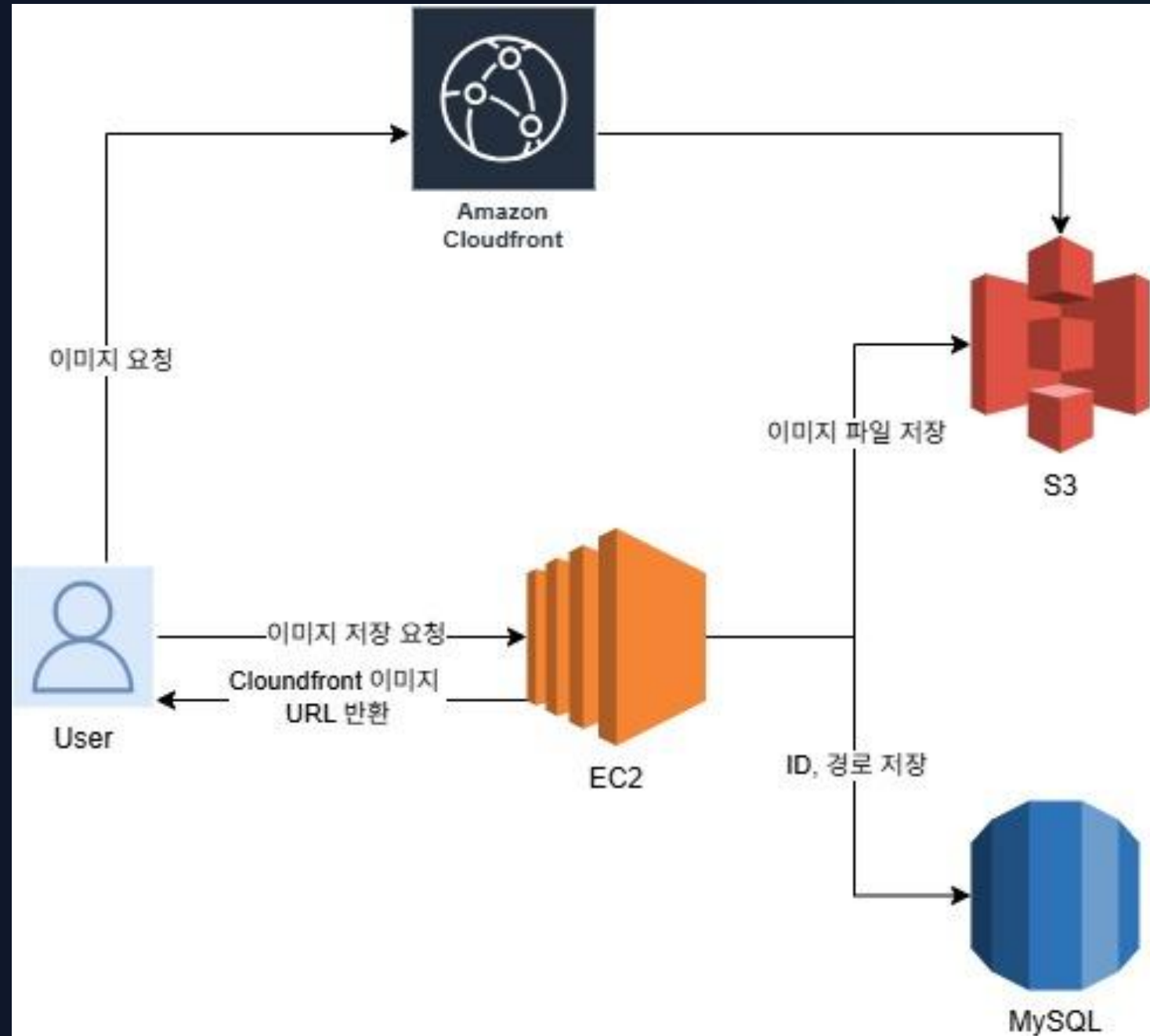
스크롤 시 이미지 추가 로딩

서버에서 이미지URL Slice로 반환

사용자가 스크롤 시 분할하여 추가

요청

이미지를 나눠서 요청하여 성능 최적화



예외처리 전략



```
public class InsufficientStockException extends BusinessException { 2 usages  👤 wlstjd50
    public InsufficientStockException( 1 usage  👤 wlstjd50 *
        String productName, int requestedQuantity, int availableStock) {
        super(String.format("상품 '%s'의 재고가 부족합니다. (요청: %d개, 남은 수량: %d개)",
            productName, requestedQuantity, availableStock),
            HttpStatus.CONFLICT);
    }
}
```

N+1 최적화

1

테스트 실행

```
@Test
@Transactional
void findProduct() {
    // given
    ProductDto product =
        createTestProduct();

    // when
    productService.findProduct
        (product.id());

    // then
    assertThat(...);
}
```

핵심 로직 실행 구간

2

SQL 로그 분석

Hibernate SQL Log

```
select p1_0.id, p1_0.name,
       p1_0.price, ...
from product p1_0
where p1_0.id=?;

select c1_0.id, c1_0.name
from category c1_0
where c1_0.id=?;

select i1_0.product_id, i1_0.url
from product_image i1_0
where i1_0.product_id=?;
... (N번 반복)
```



SQL 실행 횟수 확인
N+1 문제 발견!

3

문제 해결



Fetch Join 적용

연관 엔티티를 한 번에 조회



Batch Size 설정

IN 쿼리로 조회 횟수 감소



DTO Projection(QueryDSL)

필요한 데이터만 조회

4

최적화된 SQL

Hibernate SQL Log

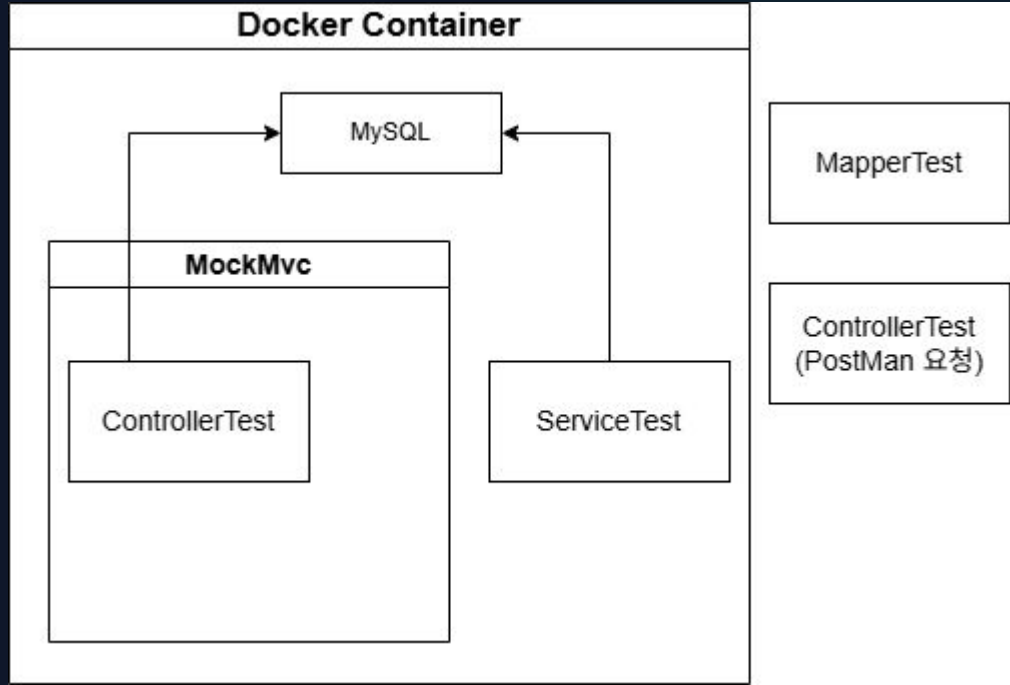
```
select p.id, p.name, p.price,
       c.name, i.url
from product p
join fetch p.category c
left join fetch p.images i
where p.id=?;
```



SQL 1번 실행
N+1 문제 해결!

테스트 코드 작성 전략

Test Layer	Target & Purpose	Environment Strategy
Controller	API endpoint 검증	Docker + MockMvc / PostMan
Service	Business logic 검증	Docker Container Test
Mapper	MapStruct 사용한 변환 검증	순수 자바 클래스 테스트



- 각 도메인마다 **Controller, Service, Mapper**에 대한 3가지 테스트 코드를 작성했습니다.
- 테스트에서 **MySQL DB**를 사용하고 운영DB와 분리된 테스트DB 환경을 위해 **Docker**를 사용했습니다.
- **Controller**는 **MockMvc**를 사용해 테스트코드로 요청과 응답을 검증하고, **PostMan**으로 실제 요청을 검증했습니다.
- **Mapper**를 사용한 **Entity ↔ DTO** 변환을 검증했습니다.

Flyway

Flyway & Migration

- ✓ DB 스키마 변경 이력 관리 자동화
- ✓ 팀원 간 로컬 DB 구조 불일치 방지하면서 구조 변경 용이
- ✓ EX)
 1. 상태 관리 단위를 Order에서 OrderItem으로 변경
 2. 상품 테이블에 이미지 컬럼 추가

resources

db

migration

V1__init.sql

V2__add_user_role.sql

V3__init_data.sql

V4__add_index_full_text.sql

V5__add_drop_order_status.sql

V6__Modify_wallets_to_set_null.sql

V7__init_data_account_record_type.sql

V8__add_account_record_type_payment.sql

V9__add_product_images.sql

V10__split_refund_account_record_type.sql

개발 환경 구축(Local)

누구나 동일한 로컬 환경에서 개발하고, 운영은 안전하게 배포합니다.

로컬 개발 환경 (Local)



연결



Docker Compose (로컬용)



Redis
Cache

docker-compose.yml

+ 장점

- ✓ 누구나 동일한 개발 환경 보장
- ✓ 환경 설정 시간 대폭 단축
- ✓ 로컬에서 실제 운영과 유사한 환경 제공

동작 흐름



docker-compose.yml
(루트 폴더)

컨테이너 실행

docker-compose up -d



MySQL 컨테이너

포트: 3306



Redis 컨테이너

포트: 6379



Spring Boot 애플리케이션
(IntelliJ에서 실행)

로컬 개발 환경 구축 순서



1. 프로젝트 Clone
GitHub에서 프로젝트
다운로드



2. Docker 컨테이너 실행
프로젝트 root에서
도커 컨테이너 실행
`docker-compose up -d`



3. Spring Boot 실행
IntelliJ에서
Spring Boot 애플리케이션 실행



4. 개발 시작
DB, Redis가 연결된
동일한 로컬 환경 완성!



핵심 정리



Docker-Compose(로컬용) 파일을
프로젝트 root에 포함



GitHub에 업로드하여
팀원 모두와 공유



어디서든 pull 후
한 번의 실행으로 동일 환경 구축



개발은 빠르게, 운영은 안전하게!
생산성 & 안정성 향상!

개발 환경 구축(배포)

설정 파일 분리 구조

application.yml (공통 설정)



- JPA, Hibernate
- Logging
- 기타 공통 설정

공통으로 사용하는 설정

application-local.yml (로컬용)



- 로컬 DB 정보
- 로컬 Redis 정보
- 기타 로컬 설정

개발 환경 전용 설정

application-prod.yml (배포용)



- 모든 값을 환경변수로 관리

```
SPRING_DATASOURCE_URL
SPRING_REDIS_HOST
AWS_ACCESS_KEY
...
```

운영 환경 전용 설정 (보안 강화)

적용 방식

- ✓ Spring Profile로
환경에 맞는 설정 자동 적용
- ✓ 민감 정보는 환경변수로 관리하여
유출 위험 최소화

Docker-Compose (배포용) 구조



docker compose



Nginx
(80, 443)



Spring Boot
(8080)



Redis
(6379)

✓ Nginx 리버스 프록시

✓ Spring Boot 실행

✓ Redis 캐시



배포용 compose 파일에는 AWS KEY, RDS 주소 등
민감 정보가 포함되어 GitHub Private Repository에 저장

구성 요소 및 저장 위치

Dockerfile 작성



Spring Boot
Dockerfile



Nginx
Dockerfile

DockerHub에 Push

GitHub Private Repository



project-prod (Private)

- docker-compose.yml
- nginx.conf

민감 정보가 포함된 compose 파일

EC2 서버



Docker + Docker Compose
설치된 EC2 인스턴스

GitHub에서 pull 후 컨테이너 실행

배포 절차 (EC2 서버)

1



GitHub에서 Pull

project-prod 저장소에서
docker-compose.yml,
nginx.conf 파일 Pull

2



compose 파일 확인

환경변수, 서비스 설정
(이미지, 포트, 볼륨 등)
확인

3



컨테이너 실행

docker compose up -d
명령어로 Nginx,
Spring Boot, Redis
컨테이너 실행

4



서비스 정상 운영

Nginx(80/443) →
Spring Boot(8080) →
Redis(6379) 연동 확인



핵심 정리



설정 파일 분리

환경별 설정 분리로 보안 강화 및
유지보수성 향상



민감 정보 보호

모든 민감 정보는 환경변수로 관리하여
유출 위험 최소화



Docker-Compose 배포

GitHub Private Repository에서 pull 후
한 번의 명령어로 간편 배포



안정적 운영

Nginx, Spring Boot, Redis를
Docker로 격리하여 안정적으로 운영

HTTPS 적용 후 403 Forbidden 오류 트러블슈팅

원인 분석부터 해결까지의 문제 해결 과정



핵심 원인

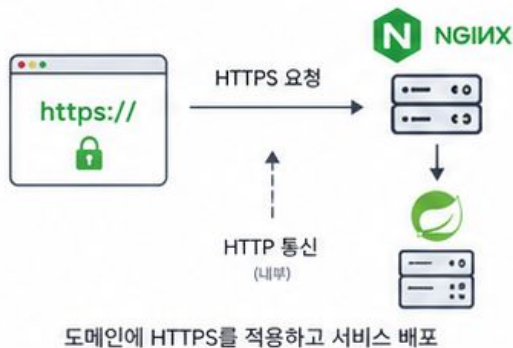
Nginx가 HTTPS 요청은 받아주지만,
Spring Boot(HTTP) 입장에서는 Origin 불일치로
Spring Security에서 요청을 차단



최종 해결

CORS 설정의 AllowedOrigins에
도메인 추가 (환경변수 활용)

1 도메인에 HTTPS 적용

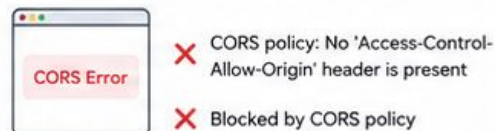


2 GET 제외한 요청에 403 Forbidden



3 CORS 문제 의심?

보통 CORS 문제라면 브라우저에서
차단되고 다음과 같은 응답이 와야 함



4 Actuator 적용 후 배포



5 Actuator로 로그 레벨 DEBUG 변경

```
$ curl -X POST "http://{EC2_IP}:8080/actuator/loggers/  
{  
  "configuredLevel": "DEBUG"  
}
```



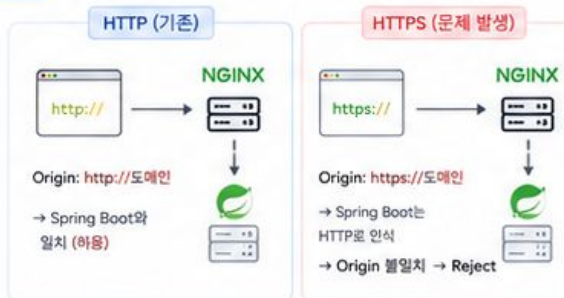
6 Spring Security 로그 확인

Spring Boot 로그에서 다음 메시지 확인

ERROR
Reject: 'https://jssdev.duckdns.org'
origin is not allowed

Spring Security에서 Origin을 허용하지 않아
요청을 거부하고 있음

7 원인 분석



핵심 원인

Nginx가 HTTPS 요청을 받아주어 브라우저는 문제가 없지만,
Spring Boot는 자신이 HTTP 서버이므로 Origin 불일치 발생

8 해결 방법

CORS 설정의 AllowedOrigins에 도메인 추가

```
@Bean  
public CorsConfigurationSource corsConfigurationSource() {  
    CorsConfiguration config = new CorsConfiguration();  
    config.setAllowedOrigins(  
        List.of(env.getProperty("cors.allowed-origins").split(","));  
    );  
    config.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE"));  
    config.setAllowedHeaders(List.of("*"));  
    config.setAllowCredentials(true);  
    UrlBasedCorsConfigurationSource source = new  
        UrlBasedCorsConfigurationSource();  
    source.registerCorsConfiguration("/*", config);  
    return source;  
}
```

환경변수 활용 예시

CORS_ALLOWED_ORIGINS=https://jssdev.duckdns.org

트러블슈팅 핵심 요약



문제

HTTPS 적용 후 GET 제외
403 Forbidden



탐색

Actuator로 로그 확인하여
Spring Security Reject 로그 발견



원인

Nginx(HTTPS) - Spring Boot(HTTP)
Origin 불일치



해결

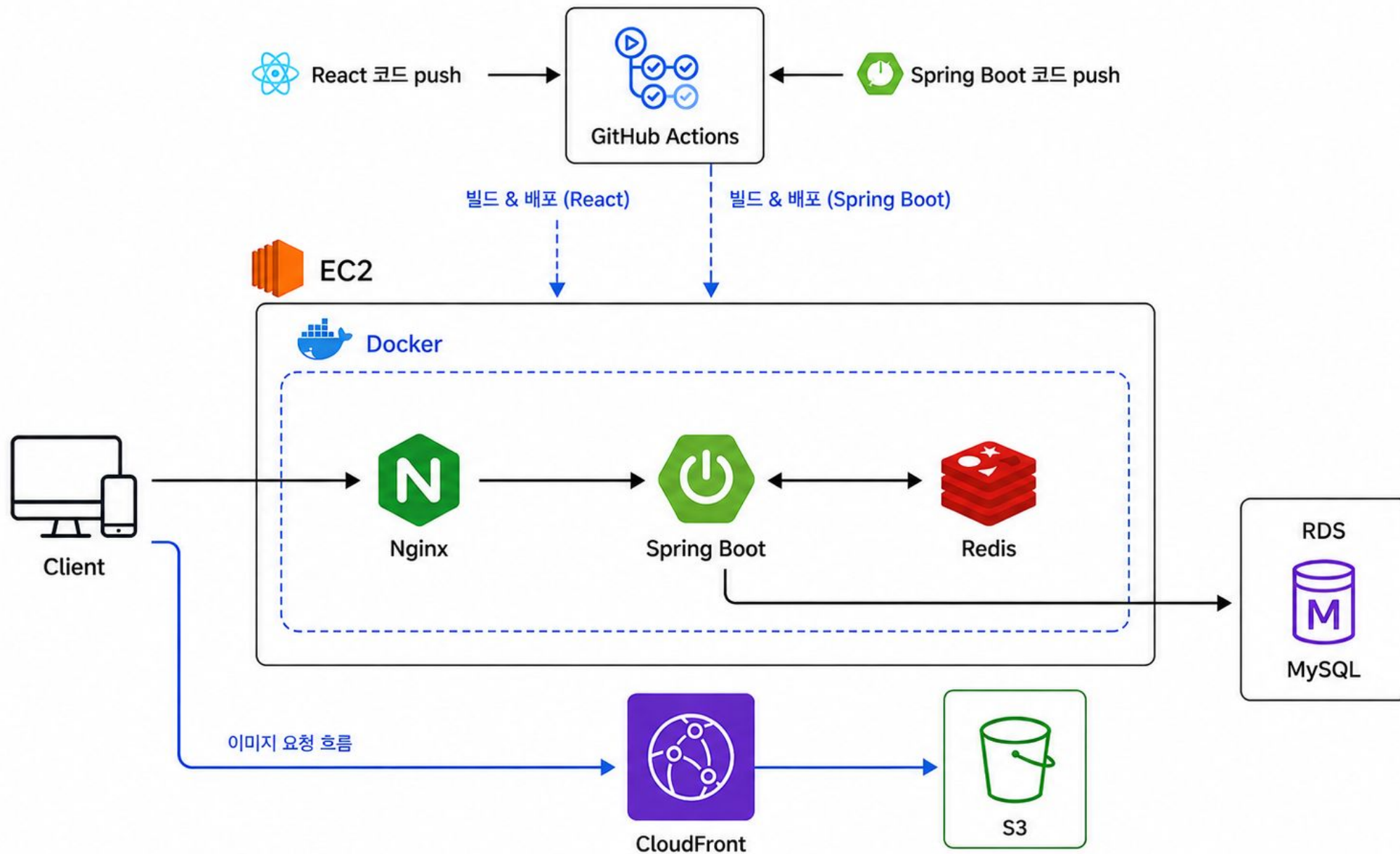
CORS AllowedOrigins에 도메인 추가
(환경변수로 관리)



결과

모든 요청 정상 동작
안정적인 서비스 운영

동작 흐름 정리



마치며

○ 목표

- 단순 기능 구현을 넘어, **N+1** 문제와 동시성 문제 등 실제 서비스 환경에서 발생할 수 있는 문제를 고려하며 개발했습니다.
- 또한 협업 상황을 가정하여 도메인 중심 설계, 책임 분리, 낮은 결합도를 유지하려 노력했으며, **Flyway, Docker, GitHub Actions**를 활용해 실제 운영과 유사한 개발 및 배포 환경을 구성했습니다.

○ 배운점

- 기본적인 인증과 스프링 **MVC**로 이어지는 흐름을 이해하고 각 도메인 중심의 서비스로직 집중, **DTO** 활용 등 서버 아키텍처와 데이터 흐름 등 전반적인 백엔드 시스템에 대한 이해를 할 수 있었습니다.
- 테스트 코드의 중요성을 배웠습니다. 새로운 기능을 추가하거나 변경할때마다 기존 테스트코드가 잘 통과되는지 검사하는 과정에서 매우 유용하고 필수적이라고 느꼈습니다.

감사합니다

GitHub Repository

https://github.com/answk427/Spring_Shop_portfolio

<https://jjsdev.duckdns.org>